Week 12: NoSQL and MongoDB CRUD

Learning outcomes: CLO1, CLO5

Lesson

NoSQL databases store data in other forms than tables:

- **document** stores (MongoDB) keep JSON-like documents;
- **key-value** stores (Redis) look up a value by its key;
- **wide-column** stores (Cassandra) group columns into column families;
- **graph** stores (Neo4j) keep nodes and the links between them.

Choose a database for how the data will be used, not for popularity. The **CAP theorem** says that when a network partition happens, a distributed database has to choose between consistency and availability.

MongoDB stores **documents** (BSON, similar to JSON) in **collections**. Documents in one collection can have different fields, and a field can hold an object or an array. `find(filter, projection)` selects documents and fields. Operators such as `$gt` and `$in` build filters, and dot notation (`"districts.district_pcode"`) reaches inside nested fields.

The MongoDB data has the same three collections as MySQL. Read from `cambodia` and make changes only in `student_work`. MongoDB adds the `_id` field; it is not part of the source data.

Worked example

Run `example.js` (see Run a MongoDB file (`setup/README.md`)) and read the output. Then open `work.js`.

Collections for this week

The MongoDB database `cambodia` has one document per row of the MySQL tables, with the same field names. MongoDB adds an `_id` to each document. Read from `cambodia` and write only to `student_work`.

Collection	Field	Type	Meaning
`province`	`province_pcode`	string	Province code
`province`	`province_name`	string	Province name (English)
`province`	`area_km2`	number	Area in km², one decimal place
`district`	`district_pcode`	string	District code
`district`	`district_name`	string	District name (English)
`district`	`area_km2`	number	Area in km², one decimal place
`district`	`province_pcode`	string	Province code
`commune`	`commune_pcode`	string	Commune code
`commune`	`commune_name`	string	Commune name (English)
`commune`	`area_km2`	number	Area in km², one decimal place
`commune`	`district_pcode`	string	District code

Documents are linked by their codes (`district.province_pcode` matches `province.province_pcode`, and `commune.district_pcode` matches `district.district_pcode`). MongoDB does not check these links the way a MySQL foreign key does.

Your collection: `example.js` copies the provinces into `student_work.w12_province` and adds a unique index on `province_pcode`. Do your CRUD practice on this copy.

Run `db.province.findOne()` in mongosh to see a document. The data guide (`data/README.md`) explains where the data comes from.

Lab tasks

1. Run `example.js` and look at one document from each collection.
2. Find the provinces with an area above a value you choose from the data, showing only code, name and area.
3. Copy the provinces into `student_work.w12_province` and create a unique index on `province_pcode`.
4. Delete one document from your copy, insert the same document again, and check that its fields match `cambodia.province`.
5. Update one name in your copy to the value it already has. Explain the difference between `matchedCount` and `modifiedCount`.

Homework

1. Write four CRUD commands, save their output, and compare each one with the matching SQL statement.
2. Explain where embedding districts inside provinces could help, and when a relational database is still the better choice.
3. In `student_work`, build one province document with an array of its districts, copied from `cambodia.district`. Query a nested field with dot notation (for example `districts.district_pcode`) and the array with `$size` or `$elemMatch`.

Capstone sprint

C12: Add a document-based part to your project and write down how your tables map to documents.

Submit

Write your answers in `work.js` and label each one, for example `// Lab 3`. Answer the questions in `reflection.md` and save your query output in `results/`. Then commit and push to your repository. See SUBMISSIONS.md for details.